RESEARCH ARTICLE

Base64 Decoding with Ignorable Characters using SIMD instructions

Daniel Lemire[1]

[1]Data Science Research Center, Université du Québec (TELUQ), Montreal, Quebec, H2S 3L5, Canada

Correspondence
Daniel Lemire, Data Science Research Center, Université du Québec (TELUQ), Montreal, Quebec, H2S 3L5, Canada
Email: daniel.lemire@teluq.ca

Funding information
Natural Sciences and Engineering Research Council of Canada, Grant Number: RGPIN-2024-03787

The base64 format is ubiquitous: it is used in emails and Web documents to encode binary data as text. By convention, base64 text often includes ignorable characters such as spaces. We introduce the first base64-decoding algorithms with support for ignorable characters designed for single-instruction-multiple-data (SIMD) instructions. Our novel functions multiply the base64 decoding speed compared to traditional methods. Our algorithms have been implemented for different hardware architectures (ARM and x64). Our approach has been widely adopted in JavaScript runtimes and Web browsers.

KEYWORDS
Binary-to-text encoding, Vectorization, Web Performance

1 | INTRODUCTION

The Internet relies primarily on text formats: email [1], JSON [2], HTML [3], XML and so forth. In this context, we routinely encode arbitrary binary data (images, fonts, cryptographic keys) as text. For this purpose, the most popular approach is to encode the binary data as base64 [4]: binary email attachments are typically encoded in base64, encryption keys are often exchanged as base64, small images are frequently embedded within HTML pages using data URLs containing base64, etc.

The base64 format represents binary data using 64 distinct characters: ASCII letters (upper and lower case) as well as the ten digits, and two additional characters ('+' and '/', or '-' and '_'). For example, the ASCII string "`Base64 Encoding`" becomes the twenty base64 characters "`QmFzZTY0IEVuY29kaW5n`". Base64 is part of the standard library of many popular programming languages such as Java, Go, C#, JavaScript, Swift, and Python.

Table 1 presents examples of base64 strings and their decoded values. One or two optional padding characters (=) may appear at the end. In several cases, the ASCII whitespace characters may appear and might be ignored: space (' '),

TABLE 1 Base64 decoding examples. Some inputs include ignorable whitespace characters: space (' '), form feed ('\f'), tab ('\t'), line feed ('\n'), and carriage return ('\r').

base string	decoded data
`Y\fW\tJ\njZ A=\r=`	`abcd`
`SGVsbG8sIFdvcmxkIQ==`	`Hello, World!`
`MTIzNDU2`	`123456`

form feed ('\f'), tab ('\t'), line feed ('\n'), and carriage return ('\r'). In the examples of Table 1, the input is an ASCII string, but any bytes can be encoded as base64, including images and cryptographic keys.

Most of our processors support single-instruction-multiple-data (SIMD) instructions, whether these processors are in our phones or in our servers. Unlike regular (or "scalar") instructions operating on single words, SIMD instructions operate on several words at once. For example, you may add a vector of 16 words with another vector of 16 words using one instruction.

Muła and Lemire [5] showed that with SIMD instructions, we can decode base64 content at gigabytes per second. Unfortunately, these fast base64 decoding functions assume that the input does not have ignorable whitespace characters. Yet whitespace characters are common in base64 inputs. For example, cryptographic keys and email attachments are typically stored using fixed-width lines, separated by line-feed characters ('\n'). Domain Name System (DNS) records are stored in zone files, where payloads are serialized as base64 segments including spaces [6]. Though it is possible to prune whitespace characters prior to decoding, Muła and Lemire [7] reported that this preprocessing step could be roughly as expensive as the base64 decoding process itself. As a superior alternative, we propose integrating multiple processing tasks in a single routine:

- The input validation: ensuring that all input characters are either base64 characters or ignorable characters (spaces).
- The removal of the ignorable characters.
- The base64 decoding itself.

Our strategy consists of loading the input and writing progressively the base64 characters—without the ignorable characters—on a small buffer. When the buffer is sufficiently filled, we decode part of it at high speed. The entire processing is done using fast SIMD instructions except maybe for the tail end of the input. Furthermore, it is portable across instruction sets (ARM NEON, AVX2, AVX-512, etc.).

2 | BASE64

ASCII stands for *American Standard Code for Information Interchange*. It is one of the oldest and most widely used character encoding standards. ASCII uses a 7-bit binary code to represent each character: each character is encoded with exactly one byte. Since there are 8 bits in a byte, the most significant bit of each byte is zero. Base64 strings are ASCII strings made of the characters in Table 2 with the possible addition of some ignorable characters, typically ASCII whitespace characters (space, form feed, tab, line feed, and carriage return). Each base64 character represents an integer value in $[0, 64)$. In other words, each character carries 6 bits of data. Thus, a sequence of four base64 characters carries 24 bits of information—it is equivalent to three bytes of data. See Fig. 1. During the encoding, given three bytes s_1, s_2, s_3 interpreted as integers in $[0, 256)$, we first construct four 6-bit values: $s_1 \div 4$, $(s_2 \div 16)$ +

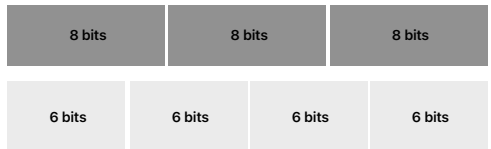


FIGURE 1 Base64 uses four characters (6-bit values) to represent any sequence of three bytes

$(s_1 \times 16) \bmod 64$, $(s_2 \times 4) \bmod 64 + (s_3 \div 64)$ and $s_3 \bmod 64$. Each of these 6-bit values corresponds to a base64 character, using Table 2. Conceptually, decoding base64 consists of converting blocks of four base64 characters back into three arbitrary byte values.

When the binary input is not divisible by three bytes, we may need to map one 8-bit value to two 6-bit values, or two 8-bit values to three 6-bit values. To make the input divisible by four characters, an encoder may add one or two padding characters ('='). This leaves either 4 or 2 bits undetermined: the excess bits are typically ignored during decoding although we can optionally require that they are zero. Thus the last block during decoding may contain 2, 3 or 4 base64 characters—but never just one character.

In 2006, the base64 format was originally specified by Josefsson [4] in the RFC 4648. The specification states that implementations must reject the encoded data if it contains characters outside the base alphabet such as whitespace characters—unless explicitly stated otherwise. In the MIME format [1] used for emails, characters outside the alphabet are ignored: no error is reported. The base64 specification defines two variants, which differ by only two characters in their alphabets: the default base64 alphabet includes '+' and '/' whereas the *URL safe* alphabet includes '–' and '_', to account for the fact that '+' and '/' have special meaning within a URL [8]. RFC 4648 warns about security issues with non-validating decoders: we find that most decoders can detect these characters and report an error. In 2016, the Web Hypertext Application Technology Working Group proposed the WHATWG Infra Standard [9]: it seeks to align standards on conventions, terminology, and data structures with respect to Web technologies. It specifies a base64 standard which, like RFC 4648, validates the input by rejecting characters outside the base alphabet, but tolerates ASCII whitespace characters. In turn, the WHATWG HTML specification [3] defines the JavaScript `atob` function in the same manner, referencing the WHATWG Infra Standard. The Ecma JavaScript standard itself includes a proposal to include Base64 support [10]. It is similar to the WHATWG Infra Standard with a few additions: the support for two alphabets (standard base64 and base64url), the ability to specify whether a padding character is used, and a decoding option regarding the last few characters (loose, strict, or stop-before-partial).

The specifications do not always require that the input be encoded using one byte per character: it may happen that the ASCII characters each occupy a 16-bit unit if they are stored as a UTF-16 Unicode string. UTF-8 and UTF-16 are two Unicode encoding formats used to represent characters in digital systems. UTF-8 is the dominant format on the Internet, although UTF-16 is the default in many programming languages (Java, JavaScript, C#). UTF-8 is a variable-length encoding where characters take between 1 and 4 bytes; it is backward-compatible with ASCII, using exactly one byte for all ASCII characters. Thus when UTF-8 is used to encode ASCII content, it is indistinguishable from pure ASCII encoding. To check that a UTF-8 string is ASCII, we only need to check that the most significant bit of each byte is zero. On the other hand, UTF-16 uses either 2 or 4 bytes per character; most commonly used characters (the Basic Multilingual Plane) are encoded in 2 bytes, while less common characters require 4 bytes through a technique called surrogate pairs. UTF-16 is less space-efficient for ASCII-dominated texts: ASCII content is stored using a two-byte word for each character. With ASCII, each 16-bit word stores a 7-bit value, so its 17 most significant bits are zero.

TABLE 2 Base64 mapping between 6-bit values and ASCII characters. The last two characters (values 62 and 63) depend on the chosen alphabet.

value	char	value	char	value l	char	value	char
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+ or -
15	P	31	f	47	v	63	/ or _

When checking that a character is ASCII in UTF-16, we have to check that these bits are zero. Thus, irrespective of the text format (ASCII, UTF-8, or UTF-16) decoding base64 data proceeds similarly.

The WHATWG forgiving base64 decoding specification requires us to produce an algorithm equivalent to the following. We begin with an input that contains a fixed number of ASCII characters (possibly in UTF-16 format).

1. Remove the ASCII whitespace characters: space (' '), form feed ('\f'), tab ('\t'), line feed ('\n'), and carriage return ('\r').
2. If the number of remaining characters is divisible by four, then remove up to two trailing padding characters (=).
3. If the input size in characters divided by four has a remainder of 1: return a failure.
4. If the input contains a character that is not part of the base64 alphabet, return a failure.
5. Starting from the beginning and as long as there are at least four characters left, look them up in Table 2 and convert them into four 6-bit values (integers in $[0, 64)$): d_1, d_2, d_3, d_4. We then output the three byte values: $(d_1 \times 4) + (d_2 \div 16), (d_2 \times 16) + (d_3 \div 4), (d_3 \times 64) + d_4$.
6. If there are three characters left, decode them from Table 2 into three 6-bit values d_1, d_2, d_3 and output two bytes: $(d_1 \times 4) + (d_2 \div 16), (d_2 \times 16) + (d_3 \div 4)$. If there are two characters left, decode them from Table 2 into three 6-bit values d_1, d_2 and output value $(d_1 \times 4) + (d_2 \div 16)$. Observe that in both cases, a few bits from the input are discarded (deliberately).

Standard libraries provide different support of base64 decoding. See Table 3. While Java adopts a strict approach,

TABLE 3 Base64 decoding conventions by programming language. A RFC 4648 decoder returns an error on non-base64 characters (e.g., spaces).

language	standard functions	behavior
ECMAScript 2024 (JavaScript)	`atob`, `Uint8Array.FromBase64`	WHATWG forgiving
.NET 9/C#	`Convert.FromBase64String`, `Base64.DecodeFromUtf8`	WHATWG forgiving
Go 1	`base64.Decode`	ignores line feed ('\n'), and carriage return ('\r') but otherwise rejects non-base64 characters
Python 3.13	`b64decode` (`validate=False`), default	error on non-ASCII, ignores other non-base64 characters
Python 3.13	`b64decode` (`validate=True`)	RFC 4648
PHP 8	`base64_decode` (`$strict = false`), default	ignores non-base64 characters
PHP 8	`base64_decode` (`$strict = true`)	RFC 4648
Java 22	`Base64.getDecoder().decode`	RFC 4648

C# and JavaScript follow the WHATWG forgiving base64 decoding approach, tolerating whitespace characters but validating the rest of the content. By default, Python and PHP ignore non-base64 characters during decoding, although they make it possible to return an error when any non-base64 character is encountered.

Using SIMD instructions often improves performance while reducing energy usage [11, 12]. OpenJDK (Java) has SIMD-accelerated functions for both base64 encoding and decoding—but it does not support spaces in the input. The PHP implementation has SIMD-accelerated functions, also for both base64 encoding and decoding—but, during decoding, if non-base64 characters are found, the function falls back on a slower scalar routine. The C# standard library (.NET) and the Node.js JavaScript runtime environment use SIMD-accelerated functions for base64 encoding. However, Node.js did not provide SIMD-accelerated functions for decoding prior to our work: they have adopted our software.

3 | OUR BASE64 DECODER: ALGORITHMIC DESIGN

Henceforth we assume that ASCII whitespace characters are ignorable. We present the main component of our algorithm as pseudocode in Fig. 2. We process the base64 input in blocks of sixty-four base64 characters. In general, there is a remainder of fewer than sixty-four base64 characters which we process using conventional (non-SIMD) code. We reserve enough space on stack for a few blocks of 64 bytes: at least 128 bytes in practice.

The caller of our base64 decoding function provides an output buffer. We assume that there is sufficient memory available on the provided output buffer. Indeed, we can efficiently compute an upper bound on the necessary output memory given an input of n characters: e.g., $\lceil n/4 \rceil \times 3$. Our decoding function itself does not allocate memory.

In a base64 stream, the padding character '=' found anywhere but at the end of the input indicates an error.

Furthermore, we should never find more than two padding characters in total. As a pre-processing step, we work backward in the input to identify up to two padding characters ('=') mixed with allowed ASCII whitespace characters. We record the number of padding characters (0, 1 or 2). We virtually remove the tail end. The rest of the processing ends right before the earliest padding character found—if one was found. There should be no more padding characters if the base64 input is valid.

In practice, input strings are either UTF-8 (including ASCII) or UTF-16. When the strings are UTF-8, we load 64 bytes of input. Otherwise, we load 128 bytes and pack them to 64 bytes with saturation: 16-bit words that do not fit in a byte are mapped to `0xFF`—and will therefore be identified as disallowed values (non-base64 characters). In all cases, we end up with 64 bytes which we attempt to keep in SIMD registers. Modern SIMD-capable processors have 16-byte registers or better. Thus, at most four SIMD registers are needed. On AVX-512 systems, a single register suffices.

In the next step, we execute a procedure to identify non-base64 characters—including allowable whitespace characters. We generate a 64-bit mask where each bit corresponds to a byte: a 0 bit indicates a valid base64 character. The 64-bit mask is zero if we only have base64 characters. This step depends on the chosen base64 alphabet. We can transform the characters as part of this step (e.g., convert them to 6-bit values). The details may vary depending on the available instruction sets.

The 64-bit mask is typically stored in a general-purpose register. We work on blocks corresponding to 64 characters because we have 64-bit general-purpose registers on 64-bit systems.

For WHATWG base64 decoding, we are required to detect invalid characters. Thus, when the 64-bit mask is non-zero, we additionally identify the allowable whitespace characters, generating a new 64-bit mask. If the two masks differ, there is an error in the input: at least one character is neither an allowable whitespace character nor a base64 character. If an error is encountered, then the processing ends. We have the option of identifying the location of the unallowed character at this stage. Optionally, we can skip this step and not check for invalid characters.

Assuming that there is no error, we continue the processing:

- If non-base64 characters were found, we write to our buffer on stack the current block of 64 bytes, while omitting the whitespace characters. We know where the non-base64 characters are thanks to the 64-bit mask we computed earlier. How the non-base64 characters are removed depends on the instruction sets available on the system. In the general case, we can use lookup tables combined with SIMD *shuffle* or *permute* instructions. The AVX-512 VBMI2 instruction set has special instructions for this purpose. In any case, between 0 and 63 bytes are written to our buffer on the stack.
- Otherwise, if only base64 characters were found, we have 64 bytes corresponding to base64 characters. If the stack buffer is empty, we can immediately decode the 64 bytes to the output memory using an efficient routine. Otherwise we store it on the stack buffer for later processing.

If the buffer has fewer than 64 bytes remaining free, we decode blocks of 64 bytes from the stack buffer to output memory. This decoding of 64 bytes can be made efficient because the content has been validated.

By our design, if the input does not contain any whitespace character, we work in uniform blocks corresponding to 64 bytes. We may expect to be able to decode an input made of pure base64 characters—with no non-base64 character—at a speed comparable to that of an algorithm that cannot skip whitespace characters.

We repeat the process as long as there are at least sixty-four characters left. At the end of the processing, we have fewer than sixty-four base64 characters left untouched in the input, and between 0 and 63 bytes in the stack buffer. We may begin by trying to fill the stack buffer so that we have a complete block of 64 bytes which can then be decoded using a fast routine. Otherwise, we handle the remainder data, the data on the stack buffer, and the remainder input characters, using conventional code. Though it might be possible to further optimize this last component, we expect

Let s by an input string of length l and an output buffer σ
$p \leftarrow 0$, $S \leftarrow$ stack buffer with a capacity of at least 128 bytes
while $p + 64 \leq l$ **do**
  $b \leftarrow (s_p, s_{p+1}, \ldots, s_{p+64-1})$
  $p \leftarrow p + 64$
  $m \leftarrow$ non-base64 (b)
  if $m \neq 0$ **then**
    $w \leftarrow$ whitespace (b)
    if $m \neq w$ **then**
      return Error
  if $m \neq 0$ **then**
    write base64 characters to S
  else
    append b to S
  if S has less than 64 bytes of remaining capacity **then**
    Given that k bytes were written to S, decode the first $\lfloor k/64 \rfloor \times 64$ bytes of S to σ
    remove $\lfloor k/64 \rfloor \times 64$ bytes from the beginning of S and move the remaining bytes to the beginning of S

FIGURE 2 Main loop of the base64 decoding algorithm. We omit the final steps for simplicity. The function non-base64 identifies the non-base64 characters in a block of 64 characters, returning a 64-bit bitset.

most base64 inputs to contain much more than 64 characters.

4 | EFFICIENT IMPLEMENTATIONS

Most commodity processors support vector or SIMD instructions (POWER, RISC-V, x64, ARM, LoongArc, etc.) but the exact instructions differ. PCs and servers often rely on x64 processors made by Intel and AMD. These processors accumulated a wide range of instruction sets: SSE2, SSE3, SSSE3, SSE 4.1, SSE 4.2, AVX, AVX2, AVX-512 (AVX512F, AVX512BW, AVX512CD, AVX512DQ, AVX512VL, …). In practice, engineers may provide a few different functions, and the software picks the best one—either at compile time or at runtime.

Mobile devices (e.g., smartphones, tablets) and some servers and laptops have 64-bit ARM processors (aarch64). These processors share the same SIMD instructions with 16-byte registers: ARM NEON (Advanced SIMD). Though there are some variations between the exact features that different 64-bit ARM processors support, it is usually convenient to design just one function that can run on all 64-bit ARM processors unless specialized features (e.g., cryptography) are needed. For base64 encoding and decoding, there is probably no need to use specialized features.

In practice, we implemented three different SIMD-based decoding functions for x64 processors—but presenting them all would be cumbersome. Instead, we choose to present the fastest implementation on x64 processors, the one that requires AVX-512 instructions. We also present the 64-bit ARM version. We make all our code freely available online.[1]

Our implementations are in C++ with function intrinsics: they are special functions which often map directly to a SIMD function. For example, the intrinsic function `vandq_u8` computes the bitwise AND between two registers.

[1] `https://github.com/simdutf/simdutf`

4.1 | ARM NEON Implementation

The ARM implementation loads blocks of 64 characters into four 16-byte registers. When the input is ASCII or UTF-8, we only require four load instructions. If the input is UTF-16, we load each register using one load instruction and two saturating-narrow-extract instructions (`uqxtn` and `uqxtn2`).

The ARM implementation transforms the loaded data during the step where we compute the mask. The function `to_base64_mask` takes a block64 structure, which refers to four 16-byte SIMD registers/variables (`b->chunks[0]`, ..., `b->chunks[3]`). It also accepts a pointer to a boolean variable where any errors during conversion can be indicated. See Fig. 3.

Initially, the function splits each byte into two 4-bit nibbles, separating the higher and lower parts. The lower nibbles are masked using a vector of 0xF to isolate the least significant bits, while the higher nibbles are obtained by shifting the bytes right by four bits.

Decoding begins by *vectorized classification* [13, 14]: it is the process by which we classify several values at once, usually by combining vectorized table lookups. We use two lookup 16-byte tables (`lut_lo` and `lut_hi`). Given an 8-bit ASCII value c, we effectively compute the classification function $f(c) = \text{lut_lo}[c \bmod 2^4] \text{ AND } \text{lut_hi}[c \div 2^4]$. The tables are designed so that:

- the result of classification function $f(c)$ is zero for any base64 character,
- the result is 1, 2 or 3 when the result is a whitespace character,
- the result is a value greater than 3 otherwise.

Thus, to check for an error, all we have to do is compute the bitwise OR of classification values and find the largest value: if it is greater than 3, then there is an error. If it is non-zero, we have non-base64 characters. In this case, we compute a 64-bit mask indicating where the non-base64 characters are. Since ARM NEON does not have a built-in instruction to turn four SIMD registers into a 64-bit mask, we compute the mask using several instructions. We compute the bitwise AND of four SIMD variables with the mask (`0x01`, `0x02`, `0x4`, `0x8`, `0x10`, `0x20`, `0x40`, `0x80`, `0x01`, `0x02`, `0x4`, `0x8`, `0x10`, `0x20`, `0x40`, `0x80`), where these values represent bits at positions 1 through 8, repeated twice, and then perform horizontal pairwise byte sums on two 16-byte vectors treated as 32 bytes, summing adjacent bytes to produce a new byte which is then written to the output; this process is repeated four times, effectively reducing the initial 64 bytes into a single 64-bit integer.

Lastly, the function includes an adjustment phase where it modifies the input data, converting the characters into the 6-bit data (see Table 2). We use a single table lookup of the form $c \rightarrow c + \text{roll_lut}[((c \div 2^4) + (c = \text{'\textbackslash'}? - 1 : 0)) \bmod 2^4] \bmod 2^8$ where $(c = \text{'\textbackslash'}? - 1 : 0)$ has value -1 when $c = \text{'\textbackslash'}$ and is zero otherwise.

If the mask is zero, we just store the result on the stack buffer. If the mask is non-zero, we *compress*—remove bytes associated with whitespace characters—the data. We compress 16-byte register by 16-byte register, writing to the stack buffer. The compress function is designed to compress a 16-byte vector (`uint8x16_t` data) into an output buffer according to a given 16-bit mask (`uint16_t` mask). If the mask is zero, the function simply copies the entire data vector to the output without any compression. For non-zero masks, it first splits the mask into two 8-bit parts (`mask1` for the least significant byte and `mask2` for the most significant byte) to handle each half of the 16-byte data separately. It uses these mask parts to look up precomputed indices for each possible byte value of the mask, to create a *shuffle vector* (`compactmask`). The necessary table is small by design (2 KiB). This shuffle vector is then adjusted by adding an offset of 8 to its second half, effectively preparing it to select and rearrange bytes from data. The function then uses SIMD the instruction `tbl` (generated by the intrinsic function `vqtbl1q_u8`) to select bytes from data based on the shuffle vector, effectively pruning the data to only those bytes that should be included according to the mask. The `tbl` instruction in ARM NEON performs vector table lookups by selecting elements from one or more source

```
uint64_t to_base64_mask(block64 *b, bool *error) {
  uint8x16_t v0f = vdupq_n_u8(0xf);
  uint8x16_t lo_nibbles0 = vandq_u8(b->chunks[0], v0f); uint8x16_t lo_nibbles1 = vandq_u8(b->chunks[1], v0f);
  uint8x16_t lo_nibbles2 = vandq_u8(b->chunks[2], v0f); uint8x16_t lo_nibbles3 = vandq_u8(b->chunks[3], v0f);
  uint8x16_t hi_nibbles0 = vshrq_n_u8(b->chunks[0], 4); uint8x16_t hi_nibbles1 = vshrq_n_u8(b->chunks[1], 4);
  uint8x16_t hi_nibbles2 = vshrq_n_u8(b->chunks[2], 4); uint8x16_t hi_nibbles3 = vshrq_n_u8(b->chunks[3], 4);
  uint8x16_t lut_lo = uint8x16_t{0x3a, 0x70, 0x70, 0x70, 0x70, 0x70, 0x70, 0x70,
                     0x70, 0x61, 0xe1, 0xb4, 0xe5, 0xe5, 0xf4, 0xb4};
  uint8x16_t lo0 = vqtbl1q_u8(lut_lo, lo_nibbles0); uint8x16_t lo1 = vqtbl1q_u8(lut_lo, lo_nibbles1);
  uint8x16_t lo2 = vqtbl1q_u8(lut_lo, lo_nibbles2); uint8x16_t lo3 = vqtbl1q_u8(lut_lo, lo_nibbles3);
  uint8x16_t lut_hi = uint8x16_t{0x11, 0x20, 0x42, 0x80, 0x8, 0x4, 0x8, 0x4,
                     0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20};
  uint8x16_t hi0 = vqtbl1q_u8(lut_hi, hi_nibbles0); uint8x16_t hi1 = vqtbl1q_u8(lut_hi, hi_nibbles1);
  uint8x16_t hi2 = vqtbl1q_u8(lut_hi, hi_nibbles2); uint8x16_t hi3 = vqtbl1q_u8(lut_hi, hi_nibbles3);
  uint8_t checks =
      vmaxvq_u8(vorrq_u8(vorrq_u8(vandq_u8(lo0, hi0), vandq_u8(lo1, hi1)),
                       vorrq_u8(vandq_u8(lo2, hi2), vandq_u8(lo3, hi3))));
  uint8x16_t bit_mask = {0x01, 0x02, 0x4, 0x8, 0x10, 0x20, 0x40, 0x80,
                         0x01, 0x02, 0x4, 0x8, 0x10, 0x20, 0x40, 0x80};
  uint64_t badcharmask = 0; *error = checks > 0x3;
  if (checks) {
    uint8x16_t test0 = vtstq_u8(lo0, hi0); uint8x16_t test1 = vtstq_u8(lo1, hi1);
    uint8x16_t test2 = vtstq_u8(lo2, hi2); uint8x16_t test3 = vtstq_u8(lo3, hi3);
    uint8x16_t sum0 = vpaddq_u8(vandq_u8(test0, bit_mask), vandq_u8(test1, bit_mask));
    uint8x16_t sum1 = vpaddq_u8(vandq_u8(test2, bit_mask), vandq_u8(test3, bit_mask));
    sum0 = vpaddq_u8(sum0, sum1); sum0 = vpaddq_u8(sum0, sum0);
    badcharmask = vgetq_lane_u64(vreinterpretq_u64_u8(sum0), 0);
  }
  uint8x16_t roll_lut = uint8x16_t{0x0, 0x10, 0x13, 0x4, 0xbf, 0xbf, 0xb9, 0xb9,
                       0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0};
  uint8x16_t vsecond_last = vdupq_n_u8(0x2f);
  uint8x16_t roll0 = vqtbl1q_u8(roll_lut, vaddq_u8(vceqq_u8(b->chunks[0], vsecond_last), hi_nibbles0));
  uint8x16_t roll1 = vqtbl1q_u8(roll_lut, vaddq_u8(vceqq_u8(b->chunks[1], vsecond_last), hi_nibbles1));
  uint8x16_t roll2 = vqtbl1q_u8(roll_lut, vaddq_u8(vceqq_u8(b->chunks[2], vsecond_last), hi_nibbles2));
  uint8x16_t roll3 = vqtbl1q_u8(roll_lut, vaddq_u8(vceqq_u8(b->chunks[3], vsecond_last), hi_nibbles3));
  b->chunks[0] = vaddq_u8(b->chunks[0], roll0); b->chunks[1] = vaddq_u8(b->chunks[1], roll1);
  b->chunks[2] = vaddq_u8(b->chunks[2], roll2); b->chunks[3] = vaddq_u8(b->chunks[3], roll3);
  return badcharmask;
}

void compress(uint8x16_t data, uint16_t mask, char *output) {
  if (mask == 0) { vst1q_u8((uint8_t *)output, data); return;}
  uint8_t mask1 = uint8_t(mask); // least significant 8 bits
  uint8_t mask2 = uint8_t(mask >> 8); // most significant 8 bits
  uint64x2_t compactmasku64 = {tables::base64::thintable_epi8[mask1],
                               tables::base64::thintable_epi8[mask2]};
  uint8x16_t compactmask = vreinterpretq_u8_u64(compactmasku64);
  const uint8x16_t off = {0, 0, 0, 0, 0, 0, 0, 0, 8, 8, 8, 8, 8, 8, 8, 8};
  compactmask = vaddq_u8(compactmask, off);
  uint8x16_t pruned = vqtbl1q_u8(data, compactmask);
  int pop1 = tables::base64::BitsSetTable256mul2[mask1];
  compactmask = vld1q_u8(tables::base64::pshufb_combine_table + pop1 * 8);
  uint8x16_t answer = vqtbl1q_u8(pruned, compactmask);
  vst1q_u8((uint8_t *)output, answer);
}

void base64_decode_block(char *out, const char *src) {
  uint8x16x4_t str = vld4q_u8((uint8_t *)src); uint8x16x3_t outvec;
  outvec.val[0] = vorrq_u8(vshlq_n_u8(str.val[0], 2), vshrq_n_u8(str.val[1], 4));
  outvec.val[1] = vorrq_u8(vshlq_n_u8(str.val[1], 4), vshrq_n_u8(str.val[2], 2));
  outvec.val[2] = vorrq_u8(vshlq_n_u8(str.val[2], 6), str.val[3]);
  vst3q_u8((uint8_t *)out, outvec);
}
```

FIGURE 3 Essential functions for base64 decoding using ARM NEON intrinsic functions for the regular base64 alphabet

vectors based on indices contained in an index vector. This operation allows for the creation of a new vector where each element corresponds to the position specified by the indices in the index vector, filling with zero if the indices are out-of-bound (e.g., greater or equal to 16). The first eight bytes are rearranged at the beginning of the first eight bytes; the second eight bytes are rearranged at the beginning of the second eight bytes. We must still combine them. The number of set bits in `mask1` is used to select another precomputed table entry, which further refines the arrangement of the pruned bytes into the final compressed form. This step ensures that the output bytes are in the correct order for the compression scheme used. Finally, the compressed data is stored in the output buffer.

When we need to go from the stack buffer to the output buffer, we use the function `base64_decode_block`. It processes 64 bytes of encoded data to produce 48 bytes of decoded binary data by leveraging SIMD operations. Initially, the input is loaded into four 16-byte vectors using the `vld4` instruction, which effectively interleaves the base64 characters into four separate streams. The decoding process then manipulates these vectors through a series of bitwise operations: left and right shifts combined with bitwise OR to reconstruct the original binary data. Specifically, each output vector is constructed by shifting and combining bits from adjacent input vectors to reverse the base64 encoding where 3 bytes of data are encoded into 4 bytes. The function uses the `orr` instruction for bitwise OR operations, the `shl` instruction for left shifts, and the `ushr` instruction for right shifts, ensuring that each 6-bit segment of the encoded data is correctly interpreted back into its 8-bit binary form. Finally, the `vstr3` instruction is used to write the decoded data back to the output buffer: it deinterleaves the data.

4.2 | AVX-512 Implementation

Our AVX-512 implementation requires specifically VBMI2. VBMI2, or Vector Byte Manipulation Instructions 2, is an instruction set architecture introduced by Intel. VBMI2 allows for efficient data manipulation at the byte level, such as compression, expansion, and permutation of bytes within vectors. By providing hardware acceleration for these operations, VBMI2 can significantly speed up tasks that would otherwise require multiple instructions. The VBMI2 instruction set is part of recent AVX-512-capable processors: AMD Zen 4 and Zen 5, Intel Ice Lake, Intel Rocket Lake, Intel Sapphire Rapids, Intel Emerald Rapids, etc. There are older AVX-512 capable processors that do not support VBMI2, but these older designs suffered from AVX-512 *licensing* issues: when 512-bit instructions were first issued, the processor would switch into a lower processor frequency. Newer processors are not similarly limited.

When the input is UTF-16, we load two 64-byte registers and we combine them using a saturating pack instruction (`vpackuswb`) which converts 16-bit values to 8-bit values. Because the result interleaves the values, we must also permute them: a single instruction (`vpermq`) suffices. The net result is that we get a single 64-byte register corresponding to the sixty-four input loaded characters so that if any character was non-ASCII, it is still non-ASCII.

In the case of our AVX-512 implementation, the 64-byte blocks fit in a single register/variable: so our type `block64` has a single data element (`chunks[0]`) containing 64 bytes. See Fig. 4 for our main functions. By comparison with the NEON implementation (Fig. 3), we require fewer lines of code and fewer instructions thanks to AVX-512 more powerful instructions and wider registers.

For vectorized classification, the AVX-512 VBMI instruction set has a powerful instruction that is ideally suited: `vpermi2b`, called by the `_mm512_permutex2var_epi8` intrinsic function [5]. This instruction uses the least significant 7 bits of each byte to look up a value among 128 bytes, stored in two 64-byte registers. The function `to_base64_mask` operates on 64-byte blocks of data. The function is invoked with a pointer to a block64 structure and a pointer to an error variable, which will be used to signal any discrepancies encountered during processing.

Two 64-byte lookup tables (`lookup0` and `lookup1`) are used for the vectorized classification. They effectively form one 128-byte lookup table. This lookup operation effectively maps each byte in the input to its base64 equivalent

```
uint64_t to_base64_mask(block64 *b, uint64_t *error) {
  __m512i input = b->chunks[0];
  const __m512i ascii_space_tbl = _mm512_set_epi8(
      0, 0, 13, 12, 0, 10, 9, 0, 0, 0, 0, 0, 0, 0, 0, 32, 0, 0, 13, 12, 0, 10,
      9, 0, 0, 0, 0, 0, 0, 0, 0, 0, 32, 0, 0, 13, 12, 0, 10, 9, 0, 0, 0, 0, 0, 0,
      0, 0, 32, 0, 0, 13, 12, 0, 10, 9, 0, 0, 0, 0, 0, 0, 0, 0, 32);
  __m512i lookup0 = _mm512_set_epi8(
        -128, -128, -128, -128, -128, -128, 61, 60, 59, 58, 57, 56, 55, 54, 53,
        52, 63, -128, -128, -128, 62, -128, -128, -128, -128, -128, -128, -128,
        -128, -128, -128, -1, -128, -128, -128, -128, -128, -128, -128, -128,
        -128, -128, -128, -128, -128, -128, -128, -128, -128, -128, -1, -128,
        -128, -1, -1, -128, -128, -128, -128, -128, -128, -128, -128, -128);
  __m512i lookup1 = _mm512_set_epi8(
        -128, -128, -128, -128, -128, 51, 50, 49, 48, 47, 46, 45, 44, 43, 42,
        41, 40, 39, 38, 37, 36, 35, 34, 33, 32, 31, 30, 29, 28, 27, 26, -128,
        -128, -128, -128, -128, -128, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16,
        15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0, -128);
  const __m512i translated = _mm512_permutex2var_epi8(lookup0, input, lookup1);
  const __m512i combined = _mm512_or_si512(translated, input);
  const __mmask64 mask = _mm512_movepi8_mask(combined) & input_mask;
  if (mask) {
    __mmask64 spaces = _mm512_cmpeq_epi8_mask(_mm512_shuffle_epi8(ascii_space_tbl, input), input) &
          input_mask;
    *error = (mask ^ spaces);
  }
  b->chunks[0] = translated;
  return mask;
}

uint64_t compress_block(block64 *b, uint64_t mask, char *output) {
  uint64_t nmask = ~mask;
  __m512i c = _mm512_maskz_compress_epi8(nmask, b->chunks[0]);
  _mm512_storeu_si512(reinterpret_cast<__m512i *>(output), c);
  return _mm_popcnt_u64(nmask);
}

void base64_decode(char *out, __m512i str) {
  __m512i merge_ab_and_bc = _mm512_maddubs_epi16(str, _mm512_set1_epi32(0x01400140));
  __m512i merged = _mm512_madd_epi16(merge_ab_and_bc, _mm512_set1_epi32(0x00011000));
  __m512i pack = _mm512_set_epi8(
      0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 60, 61, 62, 56, 57, 58,
      52, 53, 54, 48, 49, 50, 44, 45, 46, 40, 41, 42, 36, 37, 38, 32, 33, 34,
      28, 29, 30, 24, 25, 26, 20, 21, 22, 16, 17, 18, 12, 13, 14, 8, 9, 10, 4,
      5, 6, 0, 1, 2);
  __m512i shuffled = _mm512_permutexvar_epi8(pack, merged);
  _mm512_mask_storeu_epi8((__m512i *)out, 0xffffffffffff, shuffled);
}
```

FIGURE 4 Essential functions for base64 decoding using AVX-512 intrinsic functions for the regular base64 alphabet

or flags it if it is invalid (with the value `0xFF` or `-128`). When the input is valid base64 characters, the result is the 6-bit decoded value as per Table 2. Unfortunately, the `vpermi2b` instruction ignores the most significant bit of each input byte. To compensate, we compute the bitwise OR of the translated values with the input. If any non-base64 character is present, the combined result has the most significant bit set.

Subsequently, the function checks if there are any bytes in the combined result that do not match the expected base64 characters by creating a mask. If any invalid bytes are detected, an additional check is performed against a table (`ascii_space_tbl`) which maps ASCII whitespace characters, identifying the location of the whitespace characters in the base64 encoded string. The mask created from this comparison is then XORed with a mask representing spaces to determine if the error is due to non-space, non-base64 characters, setting the error variable accordingly. Finally, the function overwrites the original input with the translated data and returns the mask, which can be used for further processing or error handling in the calling context.

The function `compress_block` takes a `block64` structure, a mask indicating where the non-base64 characters, and an output buffer. It first inverts the mask to get `nmask` which indicates the bytes we want to keep. It then uses the

TABLE 4 Systems

Processor	Frequency	Microarchitecture	Memory	Compiler
Apple M2	3.0 GHz	Avalanche (aarch64, 2022)	LPDDR5 (6400 MT/s)	Apple/LLVM 16
Intel Xeon Gold 6338	3.49 GHz	Ice Lake (x64, 2019)	DDR4 (3200 MT/s)	GCC 12

intrinsic function `_mm512_maskz_compress_epi8` (and the AVX-512 VBMI2 instruction `vpcompressb`) to compress bytes from `b->chunks[0]` based on `nmask`, storing only the bytes where `nmask` bits are set. The compressed data is then stored into output, and the function returns the count of set bits in `nmask` by using the `_mm_popcnt_u64` intrinsic function (corresponding to the instruction `popcnt`), which effectively indicates how many bytes were written.

The function `base64_decode` processes the sixty-four 6-bit data. First, it multiplies and adds unsigned bytes to merge adjacent base64 characters into pairs (AB and BC) with `_mm512_maddubs_epi16`, then further combines these into a 24-bit value per 32-bit integer with another multiply-add operation using `_mm512_madd_epi16`. A predefined shuffle mask pack is used with `_mm512_permutexvar_epi8` to rearrange the bits to their correct positions in the output byte stream. Finally, the result is stored with a mask to write only the 48 valid bytes.

5 | EXPERIMENTS

We implemented our software in C++. To ease reproducibility, we make our software and our benchmarks freely available.[2] We compile our code using CMake with the default Release flags. To measure the speed, we repeatedly compute and execute the decoding functions over our test inputs so that the total time elapsed is at least one second. We compute the best speed (among all runs) and the average speed. We report the average time while verifying that the gap between the best and average speed is less than 10%. All of our software is single-threaded, and there is no network or disk access.

We summarize the characteristics of our hardware platform in Table 4. We use performance counters during benchmarking. Performance counters in CPUs are specialized registers designed to count hardware events like cache hits, branch mispredictions, executed instructions, and clock cycles, among others.

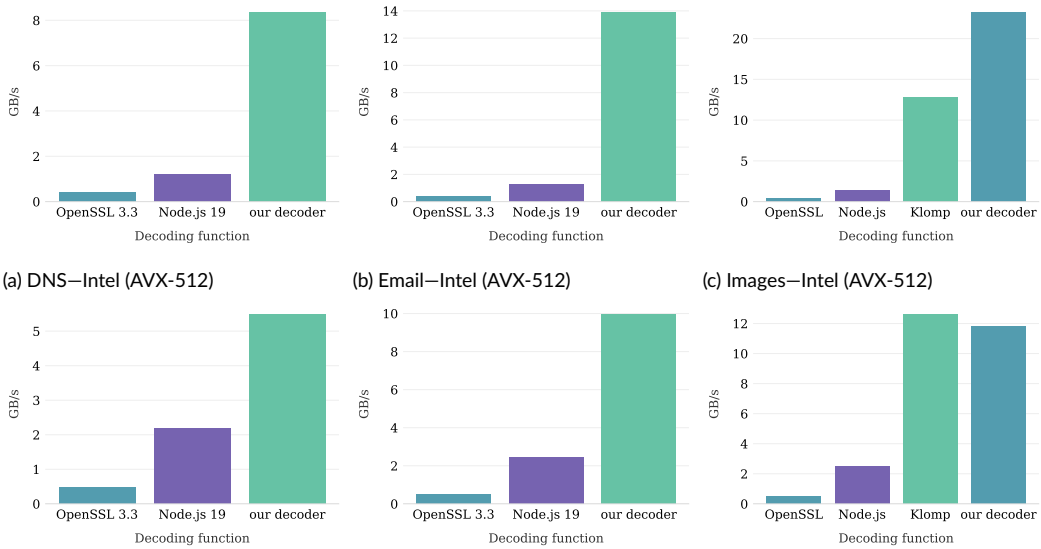
On Skylake and Cascade Lake Intel processors, AVX-512 instructions are subject to downclocking (i.e., AVX-512 *licensing*): the processor lowers its frequency systematically and significantly when AVX-512 instructions are executed. On more recent Intel processors (Ice Lake and Sapphire Rapids) as well as on AMD processors (Zen 4 and Zen 5), the effect of AVX-512 instructions on the frequency is often small or non-existent. Our software only enables AVX-512 decoding on these newer processors. On our Intel processor (Ice Lake), we only observe a small reduction of the frequency in some instances when using AVX-512 instructions.

We make all of our data available online.[3] We use base64 data contained in the Top-Level Domain (TLD) `.se` (Sweden) zone file. We use the first 100 000 base64 payloads found. Each payload is made of 350 characters. Each payload contains 7 sections of base64 characters separated by spaces. We also use fifteen base64 files from the Enron email dataset.[4] They range from 397 bytes to 1 006 349 bytes. The data is separated in lines of 76 characters. Finally, we use the base64-encoded logos for the Bing and Google sites: they span 1809 bytes and 3145 bytes respectively. They do not include ignorable characters.

[2] https://github.com/simdutf/simdutf/tree/master/benchmarks/base64

[3] https://github.com/lemire/base64data

[4] https://www.cs.cmu.edu/~enron/



(a) DNS—Intel (AVX-512) (b) Email—Intel (AVX-512) (c) Images—Intel (AVX-512)

(d) DNS—Apple (ARM NEON) (e) Email—Apple (ARM NEON) (f) Images—Apple (ARM NEON)

FIGURE 5 Decoding speed in gigabytes per second

We compare our own implementations with the following software alternatives. We use the OpenSSL base64 decoder (version 3.3.0).[5] Prior to the inclusion of our software in the popular Node.js JavaScript runtime environment, it had a conventional decoder (Node.js 19.0). We include this decoder. Both the OpenSSL and Node.js decoders are based on regular C++ code. Finally, we include Klomp's SIMD-accelerated library (version 0.5.2).[6] Klomp's library provides acceleration for several hardware architectures. Unlike our own software, it does not directly support base64 inputs with ignorable characters. Though it supports AVX-512 encoding, it does not support AVX-512 decoding (only AVX2 decoding).

We provide speed results in Fig. 5. The Klomp results are only provided for the images since they do not contain ignorable characters. On the Intel system, the Klomp speed is about half that of our speed, but it is explained by the fact that it does not use AVX-512 instructions during decoding. On the Apple system, Klomp is slightly faster by about 5%. In all cases, the OpenSSL decoding function is significantly slower. The ratio between our decoder and the Node.js 19 decoder is highest for the Email dataset (10.7 on the Intel system and 4.1 on the Apple system). The email dataset contains longer base64 sequences, giving a SIMD decoder more of an advantage. For the DNS dataset, the ratios are 6.8 on the Intel system and 2.5 on the Apple system.

To help better understand the results, we provide the performance counters in Table 5. The OpenSSL decoder uses about 37 instructions per input byte—which explains its poor speed. It is immediately clear why the Intel AVX-512 results are so good with our decoder: it uses 24 times fewer instructions than the Node.js decoder. Even though it retires fewer instructions per cycle (only 2.3 versus 5.1), it is still an order of magnitude faster than the Node.js 19 decoder. We find it interesting that the Node.js uses fewer ARM instructions per input byte on the Apple/LLVM platform than in the x64 case with GCC 12 (8.5 versus 12)—even though it is the same C++ code. Nevertheless,

[5] https://www.openssl.org

[6] https://github.com/aklomp/base64/

TABLE 5 Performance counters: retired instructions per input byte, instructions retired per cycle, and effective CPU frequency for the Email dataset

(a) Intel

name	instr./byte	instr./cycle	freq. (GHz)
OpenSSL	37	4.6	3.2
Node.js	12	5.1	3.2
our decoder (AVX-512)	0.5	2.3	3.1

(b) Apple

name	instr./byte	instr./cycle	freq. (GHz)
OpenSSL	37	5.4	3.4
Node.js	8.5	6.1	3.4
our decoder (ARM NEON)	1.7	5.1	3.4

our ARM NEON decoder uses even fewer instructions per byte (five times less, or 1.7). Overall, we find that the good results of our SIMD-based decoders are explained by the fact that they require far fewer instructions than conventional functions.

6 | CONCLUSION

Our algorithm is sufficiently simple that we were able to implement it using about 200 lines of code for each instruction set. Yet, maybe surprisingly, we find that our decoding algorithms have a performance that is comparable with SIMD-based base64 decoding functions designed for inputs without spaces. Our decoding routines, now open-source, have been integrated into popular JavaScript runtime environments (Node.js and Bun), the WebKit engine used by the Apple Safari browser, and the Chromium engine used by the Google Chrome and Microsoft Edge browsers. Our software is part of the v8 JavaScript engine.

Wider and more powerful SIMD instructions are already available on some ARM processors [15] via the SVE and SVE2 extensions. Unlike the existing ARM NEON instruction set, they support variable-length registers. Future work should examine whether better performance is possible with SVE and SVE2 than with NEON.

Another area for future work is the RISC-V processors and their vector extension (RVV). Like SVE and SVE2, it relies on a variable-length design. It would be interesting to extend our algorithm to RISC-V.

Finally, Longsoon processors have SIMD instructions (LSX and LASX) which are superficially similar to x64 SIMD instructions. However, they have many unique features such as arbitrary permutations within 4-byte lanes from an immediate value, in-register shuffling from two registers, and so forth. Future works should extend our algorithm to these instruction sets.

references

[1] Freed N, Borenstein NS, Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies; 1996. Internet Engineering Task Force, Request for Comments: 2045. https://tools.ietf.org/html/rfc2045.

[2] Bray T, The JavaScript Object Notation (JSON) Data Interchange Format; 2017. Internet Engineering Task Force, Request for Comments: 8259. https://tools.ietf.org/html/rfc8259.

[3] The Web Hypertext Application Technology Working Group, HTML Standard; 2013. https://html.spec.whatwg.org/ [last checked May 2024].

[4] Josefsson S, The Base16, Base32, and Base64 Data Encodings; 2006. Internet Engineering Task Force, Request for Comments: 4648. https://tools.ietf.org/html/rfc4648.

[5] Muła W, Lemire D. Base64 encoding and decoding at almost the speed of a memory copy. Software: Practice and Experience 2020;50(2):89–97.

[6] Koekkoek J, Lemire D. Parsing Millions of DNS Records per Second. Software: Practice and Experience 2024;To appear.

[7] Muła W, Lemire D. Faster Base64 encoding and decoding using AVX2 instructions. ACM Transactions on the Web (TWEB) 2018;12(3):1–26.

[8] Nizipli Y, Lemire D. Parsing millions of URLs per second. Software: Practice and Experience 2024;54(5):744–758.

[9] The Web Hypertext Application Technology Working Group, Infra Standard; 2016. https://github.com/whatwg/infra [last checked May 2024].

[10] ECMA, Uint8Array to/from base64; 2024. https://tc39.es/proposal-arraybuffer-base64/spec/ [last checked October 2024].

[11] Cebrian JM, Natvig L, Jahre M, Scalability analysis of AVX-512 extensions; 2019. To appear in The Journal of Supercomputing.

[12] Guermouche A, Orgerie AC. Thermal design power and vectorized instructions behavior. Concurrency and Computation: Practice and Experience 2022;34(2):e6261.

[13] Keiser J, Lemire D. Validating UTF-8 in less than one instruction per byte. Software: Practice and Experience 2021;51(5):950–964.

[14] Langdale G, Lemire D. Parsing gigabytes of JSON per second. The VLDB Journal 2019;28(6):941–960.

[15] Stephens N, Biles S, Boettcher M, Eapen J, Eyole M, Gabrielli G, et al. The ARM scalable vector extension. IEEE micro 2017;37(2):26–39.